



AL-2002: Artificial Intelligence Lab

Serial No:

Quiz 01**Total Marks: 12**Monday, 9th March, 2026**Course Instructors**

Imran Khan

Student Name: _____

Student Reg. No: _____

Section: _____

Signature: _____

Quiz 01

Q1. Here is a N Queen problem in which you try to place n queens in board on n size in a way there is no conflict between any queen. Complete the code

```
import random

def generate_board(size):
    return [random.randint(0, size - 1) for i in range(size)]

def count_conflicts(board):
    conflicts = 0
    size = len(board)
    for i in range(size):
        for j in range(i + 1, size):
            if board[i] == board[j] or abs(board[i] - board[j]) == j - i or abs(i - j) ==
abs(board[i] - board[j]):
                conflicts += 1
    return conflicts

def hill_climbing(board):
    size = len(board)
    current_conflicts = _____
    print("Current Conflicts: ", current_conflicts)
    while True:
        neighbors = []
        best_move = None
        best_conflicts = _____
        # Generate neighbors
        for i in range(size):
            for j in range(size):
                if _____:
                    temp_board = board[:]
                    temp_board[i] = j
                    neighbors.append(temp_board)
```

9 Mar, 26

AI Spring 2026



```
# Check objective value of neighbors
for neighbor in neighbors:
```

```
# If there are no neighbors with better objective value, return current board
if _____:
    break
```

Q2. This is a basic travelling salesperson problem in which a salesman must visit **N** cities exactly once and return to the starting city, while minimizing the total travel distance. You need to identify the errors and fix them and also complete the code.

```
import random
import math

# Distance matrix between cities
distances = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

# Calculate total distance of the route
def route_distance(route):
    total = 0
    for i in range(sizeof(route)):
        total += distances[route[i]][route[i+1]]
    total += distances[route[:: -1]][route[0]]
    return total

# Generate neighbor by swapping two cities
def get_neighbor(route):
    new_route = route[:]
    i, j = random.sample(range(len(route)), 2)
    new_route[i], new_route[j] = new_route[j], new_route[i]
    return new_route

def simulated_annealing():
    # Initial random route
    current_route = List(range(len(distances)))
    random.shuffle(current_route)
    current_cost = route_distance(current_route)
    temperature = 1000
    cooling_rate = 0.95

    best_route = current_route[:]
    best_cost = current_cost
```



```

while temperature > 1:

    neighbor = _____
    neighbor_cost = _____

    delta = neighbor_cost - current_cost

    # Accept better solution
    if delta < 0:
        current_route = _____
        current_cost = _____

    else:
        probability = math.exp(-delta / temperature)
        if random.random() < probability:
            current_route = neighbor
            current_cost = neighbor_cost

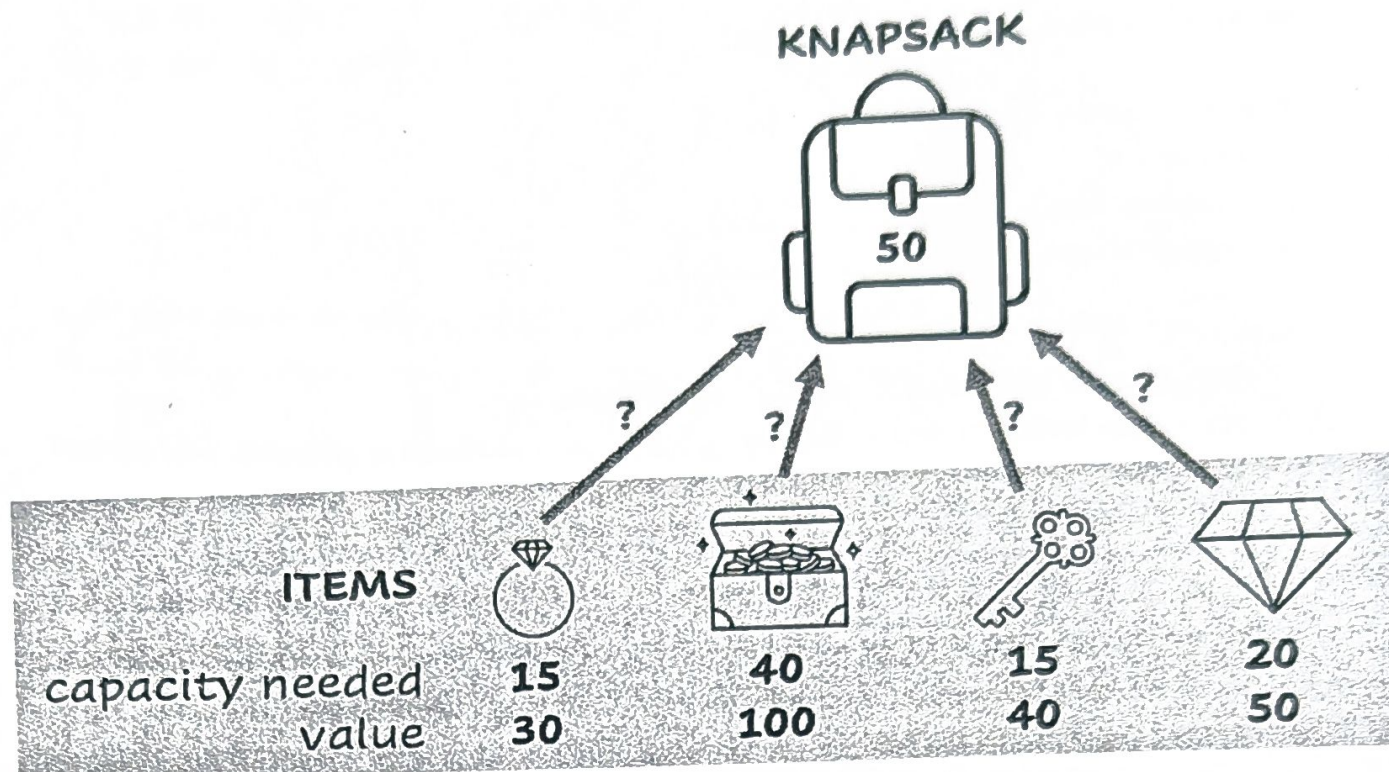
    # Update best solution
    if current_cost < best_cost:
        best_route = current_route[:]
        best_cost = current_cost

    # Cooling
    temperature -= cooling_rate

return best_route, best_cost
best_route, best_cost = simulated_annealing()
print("Best Route:", best_route)
print("Minimum Distance:", best_cost)

```

Q3. Imagine you are in a cave full of beautiful treasures. Due to some unforeseen circumstances the cave is collapsing. You have time to fill your knapsack with treasures and then you need to run away to safety. Of course, you want to take the items with you that together bring most value. What items should you take?



Identify the possible errors, fix them and complete the code

```
import copy
import random
import numpy as np
from typing import List
```

```
class Knapsack():
    def __init__(self, knapsack_capacity: int, n_items: int = 20,
                 item_values: list = None, item_capacities: list = None):

        self.name = 'knapsack'
        self.knapsack_capacity = _____

        if item_values is None and item_capacities is None:
            item_values, item_capacities = self._create_sample_data(n_items)

        self.item_values = item_values
        self.item_capacities = item_capacities
        self.n_items = len(item_values)

    def baseline_solution(self) -> list:
        capacity = 0
        solution = []
        available_items = list(range(self.n_items))
        random.shuffle(available_items)

        for _____:
            if _____ <= _____:
                _____
```



```

return solution

def score_solution(self, solution: list) -> int:
    return _____

def select_neighbor(self, solution: list) -> list:
    solution_capacity = sum(self.item_capacities[i] for i in solution)
    possible_to_add = [
        i for i in range(self.n_items)
        if _____ and
        self.item_capacities[i] <= self.knapsack_capacity - solution_capacity
    ]

    new_solution = copy.deepcopy(solution)
    # decide move
    if len(solution) == 0:
        move = 'add'
    elif len(possible_to_add) > 0:
        move = np.random.choice(['remove', 'add', 'swap'], p=[0.1, 0.6, 0.3])
    else:
        move = np.random.choice(['remove', 'swap'], p=[0.4, 0.6])

    # REMOVE
    if move == 'remove' and len(new_solution) > 0:
        new_solution.pop(random.randrange(len(new_solution)))
    # ADD
    elif move == 'add' and possible_to_add:
        new_solution.append(random.choice(possible_to_add))
    # SWAP
    elif move == 'swap' and len(new_solution) > 0:
        for _ in range(50):
            candidate = copy.deepcopy(new_solution)
            out_item = random.randrange(len(candidate))
            candidate.pop(out_item)
            in_item = random.choice(
                [i for i in range(self.n_items) if i not in candidate]
            )
            candidate.append(in_item)
            if self._is_feasible(candidate):
                new_solution = candidate
                break

    return new_solution

```